

SUY DIỄN QUY MÔ CÔNG NGHIỆP (INDUSTRIAL-STRENGTH INFERENCE)

CHƯƠNG 9.5–6, CHƯƠNG 8.1 VÀ 10.2–3

Nội dung

- ◇ Tính đầy đủ (Completeness)
- ◇ Phân giải (Resolution)
- ◇ Lập trình logic (Logic programming)

Tính đầy đủ trong FOL

Thủ tục i là đầy đủ khi và chỉ khi

$$KB \vdash_i \alpha \quad \text{mỗi khi} \quad KB \models \alpha$$

Suy diễn tiền và lùi là đầy đủ cho KB Horn
nhưng không đầy đủ cho logic bậc một tổng quát

Ví dụ, từ

$$\begin{aligned} PhD(x) &\Rightarrow HighlyQualified(x) \\ \neg PhD(x) &\Rightarrow EarlyEarnings(x) \\ HighlyQualified(x) &\Rightarrow Rich(x) \\ EarlyEarnings(x) &\Rightarrow Rich(x) \end{aligned}$$

lẽ ra phải có thể suy ra $Rich(Me)$, nhưng FC/BC sẽ không làm được điều đó

Có tồn tại một thuật toán đầy đủ không?

Lịch sử tóm tắt của việc suy luận

450TCN	Stoics	logic mệnh đề, suy diễn (có thể)
322TCN	Aristotle	“tam đoạn luận” (quy tắc suy diễn), lượng từ
1565	Cardano	lý thuyết xác suất (logic mệnh đề + độ bất định)
1847	Boole	logic mệnh đề (một lần nữa)
1879	Frege	logic bậc một
1922	Wittgenstein	chứng minh bằng bảng chân lý
1930	Gödel	$\exists$ thuật toán đầy đủ cho FOL
1930	Herbrand	thuật toán đầy đủ cho FOL (rút gọn về mệnh đề)
1931	Gödel	$\neg\exists$ thuật toán đầy đủ cho số học
1960	Davis/Putnam	thuật toán “thực tiễn” cho logic mệnh đề
1965	Robinson	thuật toán “thực tiễn” cho FOL—phân giải

Phân giải (Resolution)

Kéo theo trong logic bậc một chỉ là nửa quyết định được (semidecidable):

có thể tìm thấy một chứng minh của α nếu $KB \models \alpha$

không thể luôn luôn chứng minh rằng $KB \not\models \alpha$

So sánh với Vấn đề dừng (Halting Problem): thủ tục chứng minh có thể sắp dừng với sự thành công hoặc thất bại, hoặc có thể diễn ra mãi mãi

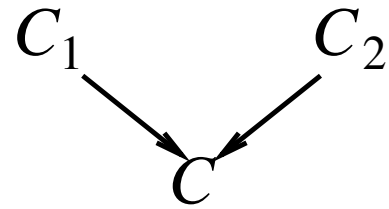
Phân giải là một thủ tục bác bỏ (refutation):

để chứng minh $KB \models \alpha$, chỉ ra rằng $KB \wedge \neg\alpha$ là không thỏa mãn được

Phân giải sử dụng $KB, \neg\alpha$ trong dạng chuẩn hội (CNF) (phép hội của các mệnh đề)

Quy tắc suy diễn phân giải kết hợp hai mệnh đề để tạo ra một mệnh đề

mới:



Sự suy diễn tiếp tục cho đến khi một mệnh đề rỗng (empty clause) được suy ra (mâu thuẫn)

Quy tắc suy diễn phân giải

Phiên bản mệnh đề cơ bản:

$$\frac{\alpha \vee \beta, \quad \neg\beta \vee \gamma}{\alpha \vee \gamma} \quad \text{hoặc tương đương} \quad \frac{\neg\alpha \Rightarrow \beta, \quad \beta \Rightarrow \gamma}{\neg\alpha \Rightarrow \gamma}$$

Phiên bản bậc một đầy đủ:

$$\frac{\begin{array}{c} p_1 \vee \dots \vee p_j \quad \dots \vee p_m, \\ q_1 \vee \dots \vee q_k \quad \dots \vee q_n \end{array}}{(p_1 \vee \dots \vee p_{j-1} \vee p_{j+1} \dots \vee p_m \vee q_1 \dots \vee q_{k-1} \vee q_{k+1} \dots \vee q_n)\sigma}$$

trong đó $p_j\sigma = \neg q_k\sigma$

Ví dụ:

$$\frac{\neg Rich(x) \vee Unhappy(x) \quad Rich(Me)}{Unhappy(Me)}$$

với $\sigma = \{x/Me\}$

Dạng chuẩn hội (Conjunctive Normal Form)

Literal = câu nguyên thủy (có thể bị phủ định), ví dụ: $\neg Rich(Me)$

Mệnh đề (Clause) = phép tuyển của các literal, ví dụ:
 $\neg Rich(Me) \vee Unhappy(Me)$

KB là phép hội của các mệnh đề

Bất kỳ FOL KB nào cũng có thể được chuyển đổi thành CNF như sau:

1. Thay thế $P \Rightarrow Q$ bằng $\neg P \vee Q$
2. Di chuyển $\neg$ vào trong, ví dụ: $\neg \forall x P$ trở thành $\exists x \neg P$
3. Chuẩn hóa biến tách biệt, ví dụ: $\forall x P \vee \exists x Q$ trở thành $\forall x P \vee \exists y Q$
4. Di chuyển các lượng từ sang trái theo thứ tự, ví dụ: $\forall x P \vee \exists x Q$ trở thành $\forall x \exists y P \vee Q$
5. Loại bỏ $\exists$ bằng phương pháp Skolemization (slide tiếp theo)
6. Bỏ các lượng từ phổ dụng
7. Phân phối $\wedge$ qua $\vee$, ví dụ: $(P \wedge Q) \vee R$ trở thành $(P \vee R) \wedge (Q \vee R)$

Phương pháp Skolemization

$\exists x \text{ Rich}(x)$ trở thành $\text{Rich}(G1)$ trong đó $G1$ là một “hằng số Skolem” mới

$\exists k \frac{d}{dy}(k^y) = k^y$ trở thành $\frac{d}{dy}(e^y) = e^y$

Khó hơn khi $\exists$ nằm bên trong $\forall$

Ví dụ: “Mọi người đều có một trái tim”

$\forall x \text{ Person}(x) \Rightarrow \exists y \text{ Heart}(y) \wedge \text{Has}(x, y)$

Sai:

$\forall x \text{ Person}(x) \Rightarrow \text{Heart}(H1) \wedge \text{Has}(x, H1)$

Đúng:

$\forall x \text{ Person}(x) \Rightarrow \text{Heart}(H(x)) \wedge \text{Has}(x, H(x))$

trong đó H là một ký hiệu mới (“hàm Skolem”)

Các đối số của hàm Skolem: tất cả các biến lượng từ phổ dụng bao quanh

Chứng minh phân giải

Để chứng minh α :

- phủ định nó
- chuyển đổi sang CNF
- thêm vào CNF KB
- suy diễn ra mâu thuẫn

Ví dụ: để chứng minh $Rich(me)$, thêm $\neg Rich(me)$ vào CNF KB

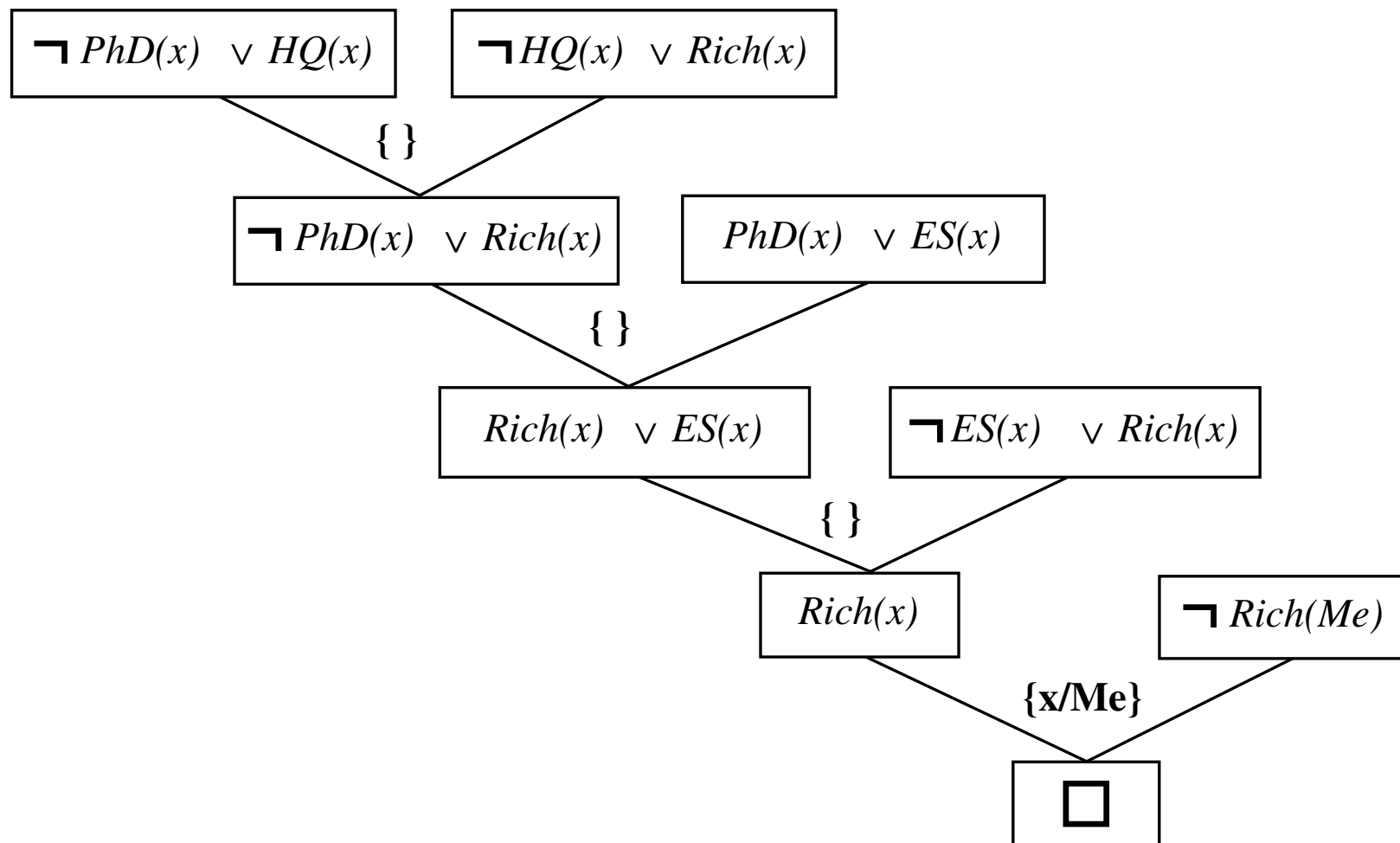
$$\neg PhD(x) \vee HighlyQualified(x)$$

$$PhD(x) \vee EarlyEarnings(x)$$

$$\neg HighlyQualified(x) \vee Rich(x)$$

$$\neg EarlyEarnings(x) \vee Rich(x)$$

Chứng minh phân giải



Lập trình logic

Khẩu hiệu: tính toán là sự suy diễn trên các KB logic

Lập trình logic	Lập trình thông thường
1. Xác định vấn đề	Xác định vấn đề
2. Thu thập thông tin	Thu thập thông tin
3. Nghĩ giải pháp	Tìm ra giải pháp
4. Mã hóa thông tin vào KB	Lập trình giải pháp
5. Mã hóa ví dụ vấn đề thành sự kiện	Mã hóa ví dụ vấn đề thành dữ liệu
6. Hỏi các truy vấn	Áp dụng chương trình vào dữ liệu
7. Tìm các sự kiện sai	Gỡ lỗi các lỗi thủ tục

Nên dễ dàng gỡ lỗi *Capital(NewYork,US)* hơn $x := x + 2$!

Hệ thống Prolog

Cơ sở: suy diễn lùi với các mệnh đề Horn + các tính năng bổ sung (bells & whistles)

Được sử dụng rộng rãi ở Châu Âu, Nhật Bản (cơ sở của dự án Thế hệ thứ 5)

Kỹ thuật biên dịch $\Rightarrow$ 10 triệu LIPS

Chương trình = tập các mệnh đề = **head** :- **literal**₁, ...
literal_n.

Hợp nhất hiệu quả bằng mã hóa mở (open coding)

Truy xuất hiệu quả các mệnh đề khớp bằng liên kết trực tiếp

Suy diễn lùi từ trái sang phải, ưu tiên chiều sâu

Các vị từ tích hợp cho số học v.v., ví dụ: **X is Y*Z+3**

Giả định thế giới đóng (“phủ định như là sự thất bại”)

ví dụ: **not PhD(X)** thành công nếu **PhD(X)** thất bại

Ví dụ Prolog

Tìm kiếm ưu tiên chiều sâu từ trạng thái bắt đầu **X**:

```
dfs(X) :- goal(X) .  
dfs(X) :- successor(X,S),dfs(S) .
```

Không cần lặp qua **S**: **successor** thành công với mỗi trường hợp

Nối hai danh sách để tạo ra danh sách thứ ba:

```
append([],Y,Y) .  
append([X|L],Y,[X|Z]) :- append(L,Y,Z) .
```

```
query:    append(A,B,[1,2]) ?
```

```
answers:  A=[]      B=[1,2]
```

```
          A=[1]     B=[2]
```

```
          A=[1,2]   B=[]
```